UART Communication

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

Contents

1 Application-Side Technical Documentation **5**
1.1 Introduction 5

2 Objectives **5**

3 Scope **5**
3.1 UART Communication Fundamentals 6

4 UART Communication Architecture **6**

5 UART Signal Connections **6**

6 UART Communication Characteristics **7**
6.1 UART Communication Parameters 7

7 Baud Rate **7**

8 Data Bits **7**

9 Parity **8**

10 Stop Bits **8**

11 Flow Control **8**

12 Common UART Configuration **8**
12.1 Linux Serial Interface Identification 9

13 Common Serial Device Names **9**

14 List Available Serial Interfaces **9**

15 Check Serial Device Information **9**

16 Check Kernel Messages **10**

17 Check Serial Device Availability **10**
17.1 UART Hardware Connection Requirements 10

18 TX, RX, and GND Connections **10**

19 Voltage-Level Compatibility **10**

20 Hardware Flow Control Connections **11**

21 UART and RS-485 Differences **11**
21.1 Serial Port Configuration in Linux 11

22 Check Existing Serial Configuration **12**

23 Configure the Serial Port Using stty **12**

24 Verify the Configuration **12**

25 Check Device Permissions **13**

26 Check Whether Another Application Is Using the Port **13**
26.1 Python Environment and PySerial Installation 13

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

27 Verify Python Installation **13**

28 Install PySerial **14**

29 Verify PySerial Installation **14**

30 Import the Serial Module **14**
30.1 UART Data Transmission and Reception Architecture 14

31 Communication Architecture **14**

32 Data Flow **15**

33 Application Message Format **15**
33.1 UART Transmitter Application 15

34 Create *uart_sender.py* **15**

35 Complete Python Code **16**

36 Explanation of Transmitter Configuration **18**
36.1 UART Receiver Application 19

37 Create *uart_receiver.py* **19**

38 Complete Python Code **20**

39 Explanation of Receiver Configuration **22**
39.1 Application Deployment and Execution 23

40 Application Directory Structure **23**

41 Verify the Serial Interface **23**

42 Verify UART Configuration **23**

43 Configure the Hardware **24**

44 Start the Receiver **24**

45 Start the Transmitter **24**

46 Stop the Applications **24**
46.1 Expected Output and Communication Validation 24

47 Transmitter Output **24**

48 Receiver Output **25**

49 Communication Validation Checklist **25**
49.1 UART Loopback Testing 26

50 Loopback Connection **26**

51 Loopback Test Using Python **26**

52 Execute the Loopback Test **27**
52.1 UART Communication Monitoring 28

53 Check Serial Configuration **28**

54 Check Serial Device Ownership **28**

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

55 Identify Processes Using the Port **28**

56 Monitor Serial Data Using a Terminal Utility **28**

57 Inspect Kernel Messages **28**
57.1 Error Handling and Communication Reliability 29

58 Serial Port Not Found **29**

59 Permission Denied **29**

60 Incorrect Received Data **29**

61 Communication Timeout **29**

62 Invalid Message Format **30**

63 Buffer and Data Loss Considerations **30**
63.1 Troubleshooting 31

64 Useful Diagnostic Commands **31**
64.1 Security and Operational Considerations 31
64.2 Conclusion 32
64.3 References 32

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

1 Application-Side Technical Documentation

Document Category: Application Development
Platform: Linux-Based Embedded Systems
Communication Interface: UART (Universal Asynchronous Receiver/Transmitter)
Communication Type: Serial Communication
Programming Language: Python 3
Application: Serial Data Transmission and Reception

1.1 Introduction

UART (Universal Asynchronous Receiver/Transmitter) is a serial communication interface commonly used in embedded systems to exchange data between a processor and external devices.

UART communication is widely used for connecting sensors, communication modules, microcontrollers, debugging interfaces, GPS receivers, wireless modules, and other serial devices.

Unlike parallel communication, UART transmits data one bit at a time through a transmit line (TX) and receives data through a separate receive line (RX). It does not require a shared clock signal because both communicating devices use predefined communication settings, such as baud rate, data bits, parity, and stop bits.

In Linux-based embedded systems, UART interfaces are generally exposed through serial device files such as */dev/ttyS0*, */dev/ttyS1*, or other platform-specific names. Applications can access these interfaces to transmit and receive data.

This document explains how to identify a supported serial port, configure UART communication, and exchange application data with an external device using Python.

The implementation demonstrates serial communication using the PySerial library. It includes port configuration, data transmission, data reception, message validation, response handling, and troubleshooting procedures.

The procedures are designed to be adaptable across different Linux-based embedded platforms without depending on a specific processor, development board, or UART controller.

2 Objectives

The main objectives of UART communication are:

- Identify available serial interfaces in Linux.
- Understand UART communication parameters.
- Configure a supported serial port.
- Establish communication with an external device.
- Transmit application data through UART.
- Receive and process serial data.
- Validate communication using a loopback test.
- Handle communication errors and timeouts.
- Monitor serial communication status.
- Troubleshoot common UART communication problems.

3 Scope

This document focuses on application-level UART communication using Linux and Python 3.

It covers:

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

- UART communication fundamentals.
- Serial interface identification.
- UART wiring and connection requirements.
- Serial-port configuration.
- Python-based UART transmission and reception.
- Data formatting and validation.
- Communication testing.
- Error handling and troubleshooting.

The examples use a standard UART connection with configurable communication parameters. The actual serial device name and supported settings depend on the target Linux platform and connected hardware.

3.1 UART Communication Fundamentals

UART is an asynchronous serial communication method used to exchange data between two devices.

In asynchronous communication, the transmitting and receiving devices do not share a clock signal. Instead, both devices must use compatible communication settings to correctly interpret transmitted data.

A typical UART connection uses two primary signal lines:

- **TX (Transmit):** Sends data from one device to another.
- **RX (Receive):** Receives data from the connected device.

A common UART connection also requires a shared ground reference.

4 UART Communication Architecture

The following architecture illustrates a basic UART connection between a Linux-based embedded device and an external device.

Linux-Based Device -> UART TX -> External Device RX

Linux-Based Device <- UART RX <- External Device TX

Linux GND <-> External Device GND

The transmitting device sends serial data through its TX line. The receiving device reads the data through its RX line.

For bidirectional communication, both TX and RX connections are required.

5 UART Signal Connections

Linux device signal	External device signal	Function
TX	RX	Transmit data to the external device
RX	TX	Receive data from the external device
GND	GND	Common electrical reference

Important: TX must be connected to RX, and RX must be connected to TX. Connecting TX to TX or RX to RX will not establish the expected communication.

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

6 UART Communication Characteristics

UART has the following characteristics:

- Asynchronous serial communication.
- Separate transmit and receive signal lines.
- No shared clock signal.
- Configurable baud rate.
- Configurable data bits, parity, and stop bits.
- Supports full-duplex communication when separate TX and RX lines are available.
- Commonly used for short-distance communication between electronic devices.

UART communication is generally point-to-point. Connecting multiple devices to the same UART signal lines requires additional hardware or a communication architecture specifically designed for that purpose.

6.1 UART Communication Parameters

Both communicating devices must use compatible serial communication settings.

If the settings do not match, the receiving device may interpret the transmitted data incorrectly or receive unreadable characters.

The main UART configuration parameters are described below.

7 Baud Rate

Baud rate defines the signaling rate used for serial communication.

Common UART baud rates include:

Baud rate	Typical application
9600	Basic serial devices and sensors
19200	Serial communication with supported external devices
38400	General serial communication
57600	Supported medium-speed serial communication
115200	Common embedded Linux communication
230400	Higher-speed serial communication when supported
460800	Higher-speed serial communication when supported

For example, a configuration of *115200* means that the serial interface operates at 115200 baud.

The actual supported baud rates depend on the UART controller, Linux driver, connected device, and communication requirements.

8 Data Bits

Data bits define the number of data bits transmitted in each UART frame.

Common configurations use 7 or 8 data bits.

For most embedded applications, 8 data bits are commonly used.

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

9 Parity

Parity provides a basic mechanism for detecting certain bit errors.

The available parity configurations are:

Parity	Description
None	No parity bit is transmitted.
Even	The parity bit is selected to make the total number of 1 bits even.
Odd	The parity bit is selected to make the total number of 1 bits odd.

Parity must be configured consistently on both communicating devices.

Parity is not a replacement for application-level error detection or a checksum.

10 Stop Bits

Stop bits indicate the end of a transmitted UART frame.

Common configurations use one or two stop bits.

11 Flow Control

Flow control regulates data transmission to prevent a receiver from being overwhelmed.

Two common types are:

No Flow Control

- No additional flow-control signals are used.
- Suitable for basic TX/RX communication when both devices can handle the data rate.

Hardware Flow Control

- Uses RTS and CTS signals.
- Allows devices to coordinate transmission.
- Requires additional physical connections and support from both devices.

Software flow control may also be available through XON/XOFF characters, depending on the device and application.

12 Common UART Configuration

A commonly used configuration is:

Parameter	Configuration
Baud rate	115200
Data bits	8
Parity	None
Stop bits	1
Flow control	Disabled
Communication	Full duplex
Data format	UTF-8 text

This configuration is commonly represented as **115200 8N1**.

It is an example configuration, not a universal requirement. Always use the settings specified by the external device.

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

12.1 Linux Serial Interface Identification

Linux exposes supported serial interfaces through device files.

Applications communicate with these device files instead of directly accessing UART controller registers.

The actual device name depends on the Linux platform, hardware configuration, connected interface, and kernel driver.

13 Common Serial Device Names

Device file	Description
`/dev/ttyS0`	Common name for a hardware serial port
`/dev/ttyS1`	Another hardware serial port
`/dev/ttyAMA0`	Serial interface name used on some ARM-based platforms
`/dev/ttyUSB0`	Serial interface provided by a USB-to-UART adapter
`/dev/ttyUSB1`	Another USB-to-serial interface
`/dev/ttyACM0`	Serial interface exposed by a USB CDC ACM device

The device names are platform-dependent. Not every Linux system provides all these interfaces.

A */dev/ttyUSB** interface generally represents a USB-to-serial adapter. A */dev/ttyACM** interface generally represents a USB CDC ACM device. These are serial interfaces exposed through USB rather than direct hardware UART device nodes.

14 List Available Serial Interfaces

Execute:

```
ls /dev/tty*
```

This displays the available terminal and serial device files.

To identify common serial interfaces:

```
ls /dev/ttyS*
```

```
ls /dev/ttyUSB*
```

```
ls /dev/ttyACM*
```

If a pattern does not match any available device, the shell may display an error. This does not necessarily indicate a system problem; that interface may simply be unavailable.

15 Check Serial Device Information

To inspect a particular device:

```
ls -l /dev/ttyS0
```

Replace */dev/ttyS0* with the actual serial interface.

Example output:

```
crw-rw---- 1 root dialout 4, 64 Oct 30 10:00 /dev/ttyS0
```

The output provides information about the device file, ownership, permissions, and group.

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

16 Check Kernel Messages

Linux kernel messages can help identify serial interfaces and connected USB-to-serial adapters.

Execute:

```
dmesg | grep -i tty
```

For USB serial devices:

```
dmesg | grep -i usb
```

Example output:

```
usb 1-1: USB Serial Device connected
usb 1-1: converter now attached to ttyUSB0
```

The actual output depends on the hardware and installed drivers.

17 Check Serial Device Availability

Before starting the application, verify that the selected device file exists:

```
test -e /dev/ttyS0 && echo "Serial port available"
```

If the device is unavailable, check the configured interface name and whether the required hardware and driver are present.

17.1 UART Hardware Connection Requirements

Before configuring the application, ensure that the electrical connection between the Linux device and the external device is suitable.

UART signal compatibility is important because different interfaces can use different voltage levels and electrical standards.

18 TX, RX, and GND Connections

For a basic three-wire UART connection:

Linux device	External device
TX	RX
RX	TX
GND	GND

The connection supports bidirectional data exchange.

If only one-way communication is required, only the corresponding transmit and receive connection may be necessary, but a shared ground reference is normally required.

19 Voltage-Level Compatibility

UART is a communication method, not a universal electrical voltage standard.

Different devices may use different electrical interfaces, including:

- 1.8 V or 3.3 V logic-level UART.
- 5 V logic-level UART.

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

- RS-232 serial signaling.
- RS-485 differential signaling.

These electrical interfaces are not directly interchangeable.

For example, a 3.3 V logic-level UART must not be connected directly to an RS-232 interface. RS-232 uses different voltage levels and polarity.

Use an appropriate level shifter or interface transceiver whenever the electrical standards differ.

Important: Verify the voltage specifications of both devices before connecting TX and RX. An incorrect voltage connection may damage the UART interface or connected hardware.

20 Hardware Flow Control Connections

If the external device requires RTS/CTS hardware flow control, the corresponding signals must also be connected.

Linux device signal	External device signal
TX	RX
RX	TX
RTS	CTS
CTS	RTS
GND	GND

The exact connection and signal direction must be verified against the hardware documentation.

Do not enable hardware flow control in software unless the required signals are physically connected and supported by both devices.

21 UART and RS-485 Differences

UART defines asynchronous serial data framing. RS-485 defines an electrical signaling standard commonly used for differential serial communication.

An RS-485 application may use a UART controller together with an RS-485 transceiver.

For RS-485 communication, additional requirements may include:

- Differential A/B signal connections.
- Correct bus termination.
- Biasing where required.
- Transmit/receive direction control.
- Appropriate bus topology.

The direct TX/RX wiring example in this document applies to compatible logic-level UART interfaces. It should not be used as an RS-485 wiring diagram.

21.1 Serial Port Configuration in Linux

Before transmitting or receiving data, the serial interface must be configured with the correct communication parameters.

The configuration includes:

- Serial device path.
- Baud rate.
- Data bits.

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

- Parity.
- Stop bits.
- Flow control.
- Read timeout.

The same settings must be used by both communicating devices.

22 Check Existing Serial Configuration

If the *stty* utility is available, execute:

```
stty -F /dev/ttyS0 -a
```

This displays the current serial-port settings.

Example:

```
speed 115200 baud;
cs8
- parenb
-cstopb
-crtscts
```

The output indicates the configured speed and selected serial parameters.

23 Configure the Serial Port Using stty

For a common 115200 8N1 configuration without hardware flow control:

```
stty -F /dev/ttyS0 115200 cs8 -parenb -cstopb -crtscts
```

Parameter descriptions:

Parameter	Meaning
`115200`	Configures the baud rate.
`cs8`	Selects 8 data bits.
`-parenb`	Disables parity.
`-cstopb`	Selects one stop bit.
`-crtscts`	Disables RTS/CTS hardware flow control.

Note: This command configures the selected serial interface for the current session. It does not necessarily make the configuration persistent after reboot.

The actual configuration method may depend on the Linux distribution and serial-port driver.

24 Verify the Configuration

Execute:

```
stty -F /dev/ttyS0 -a
```

Confirm that the output reflects the required baud rate and framing settings.

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

25 Check Device Permissions

A Linux serial device is protected by file permissions.

Inspect its ownership:

```
ls -l /dev/ttyS0
```

If the device belongs to the *dialout* group, check the current user's groups:

```
groups
```

If access is required, an administrator can add the application user to the appropriate group:

```
sudo usermod -aG dialout "$USER"
```

Log out and log in again for the new group membership to take effect.

The group name may differ between Linux distributions. Use the group shown by the serial device's ownership information.

Security note: Avoid changing serial-device permissions to unrestricted access. Grant access only to the users or services that require it.

26 Check Whether Another Application Is Using the Port

A serial interface should not normally be accessed by multiple independent applications at the same time.

Check whether another process has opened the device:

```
sudo fuser -v /dev/ttyS0
```

If the command is available, it displays processes using the specified device.

Also check whether a serial console or login service is configured on that interface. A console service may occupy the UART and interfere with application communication.

Do not disable an active console until its purpose and recovery access have been confirmed.

26.1 Python Environment and PySerial Installation

Python provides a convenient way to access serial interfaces through the PySerial library.

PySerial provides functions to:

- Open and close serial ports.
- Configure baud rate and framing.
- Transmit data.
- Receive data.
- Configure timeouts.
- Control hardware flow settings.
- Handle serial communication exceptions.

27 Verify Python Installation

Execute:

```
python3 --version
```

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

Confirm that Python 3 is available.

28 Install PySerial

On Ubuntu or Debian-based systems, PySerial can be installed using the package manager:

```
sudo apt update
sudo apt install python3-serial -y
```

Alternatively, install it using pip in an appropriate Python environment:

```
python3 -m pip install pyserial
```

On externally managed Python installations, use the distribution package or a virtual environment rather than forcing a system-wide pip installation.

29 Verify PySerial Installation

Execute:

```
python3 -c "import serial; print(serial.__version__)"
```

If the installation is successful, the command displays the installed PySerial version.

30 Import the Serial Module

Python applications can access PySerial using:

```
import serial
```

The serial module provides the interface required to communicate with the Linux serial device.

30.1 UART Data Transmission and Reception Architecture

The application uses two Linux-based devices connected through a compatible UART interface.

One application transmits application data, while the other receives the data and returns an acknowledgement.

31 Communication Architecture

Device A - UART Transmitter

- Opens the configured serial port.
- Configures UART communication parameters.
- Generates application data.
- Converts the data into a defined message format.
- Transmits the message through UART TX.
- Waits for a response from the external device.

Device B - UART Receiver

- Opens the configured serial port.
- Uses matching communication settings.
- Receives serial data.

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

- Identifies complete messages.
- Validates the received data.
- Sends an acknowledgement through UART TX.

32 Data Flow

Device A Application -> UART TX -> Device B UART RX -> Device B Application

The response follows the reverse direction:

Device B Application -> UART TX -> Device A UART RX -> Device A Application

33 Application Message Format

For this example, JSON is used to represent application data.

Example message:

```
{
    "device": "Linux_Device_01",
    "temperature": 28.5,
    "humidity": 62.0,
    "status": "normal"
}
```

A newline character (`\n`) is appended to each message.

This provides a simple message boundary so the receiver can identify where one message ends.

UART transfers a continuous stream of bytes. It does not automatically identify application message boundaries. Therefore, the application must define a framing method.

Other applications may use fixed-length messages, length fields, delimiters, or binary protocols depending on their requirements.

33.1 UART Transmitter Application

The transmitter application sends structured application data through the configured serial interface.

The example uses the following settings:

Parameter	Value
Serial port	`/dev/ttyS0`
Baud rate	115200
Data bits	8
Parity	None
Stop bits	1
Flow control	Disabled
Transmission interval	5 seconds
Message format	JSON

Replace */dev/ttyS0* with the actual serial interface available on the target system.

34 Create *uart_sender.py*

Create a Python file named *uart_sender.py*.

The application performs the following operations:

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

1. Opens the serial port.
2. Configures the UART parameters.
3. Generates application data.
4. Converts the data into JSON.
5. Adds a newline delimiter.
6. Transmits the message.
7. Waits for an acknowledgement.
8. Displays the communication result.

35 Complete Python Code

```python
#!/usr/bin/env python3

"""
UART transmitter application.

Sends JSON application data through a Linux
serial interface and receives an acknowledgement.
"""

import json
import time
import serial

# UART configuration

SERIAL_PORT = "/dev/ttyS0"

BAUD_RATE = 115200

TIMEOUT = 2

INTERVAL = 5

def generate_application_data():
    """
    Generate example application measurements.

    Replace this function with the actual
    application data acquisition method.
    """

    return {
        "device": "Linux_Device_01",
        "temperature": 28.5,
        "humidity": 62.0,
        "status": "normal"
    }

def send_data(uart, data):
    """Convert application data to JSON and transmit it."""

    message = json.dumps(data) + "\n"
```

(continues on next page)

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

(continued from previous page)

```
    encoded_message = message.encode("utf-8")

    uart.write(encoded_message)

    uart.flush()

    print("\nTransmitted data:")

    print(json.dumps(data, indent=4))

def receive_acknowledgement(uart):
    """Read one newline-terminated acknowledgement."""

    response = uart.readline()

    if not response:

        print("No acknowledgement received.")

        return

    try:

        decoded_response = response.decode(
            "utf-8"
        ).strip()

        acknowledgement = json.loads(
            decoded_response
        )

        print("\nReceived acknowledgement:")

        print(
            json.dumps(
                acknowledgement,
                indent=4
            )
        )

    except (UnicodeDecodeError, json.JSONDecodeError):

        print(
            "Received invalid acknowledgement:",
            response
        )

def main():

    try:

        with serial.Serial(
            port=SERIAL_PORT,
            baudrate=BAUD_RATE,
            bytesize=serial.EIGHTBITS,
```

(continues on next page)

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

(continued from previous page)

```
            parity=serial.PARITY_NONE,
            stopbits=serial.STOPBITS_ONE,
            timeout=TIMEOUT,
            xonxoff=False,
            rtscts=False,
            dsrdtr=False
        ) as uart:

            print("UART transmitter started.")

            print("Serial port:", SERIAL_PORT)

            print("Baud rate:", BAUD_RATE)

            print("Waiting for communication...")

            while True:

                application_data = (
                    generate_application_data()
                )

                send_data(
                    uart,
                    application_data
                )

                receive_acknowledgement(uart)

                time.sleep(INTERVAL)

    except serial.SerialException as error:

        print("UART communication error:", error)

    except KeyboardInterrupt:

        print("\nUART transmitter stopped.")

if __name__ == "__main__":

    main()
```

36 Explanation of Transmitter Configuration

Serial device selection

```
SERIAL_PORT = "/dev/ttyS0"
```

Defines the Linux serial device used by the application.

Change this value according to the serial interface available on the target system.

Baud rate

```
BAUD_RATE = 115200
```

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

Configures the serial communication speed.

The receiving device must use the same baud rate.

Opening the serial interface

```
with serial.Serial(
    port=SERIAL_PORT,
    baudrate=BAUD_RATE,
    bytesize=serial.EIGHTBITS,
    parity=serial.PARITY_NONE,
    stopbits=serial.STOPBITS_ONE,
    timeout=TIMEOUT
) as uart:
```

Opens the serial device and configures its communication parameters.

The *with* statement ensures that the serial port is closed when the application exits the block.

Generating application data

```
application_data = generate_application_data()
```

Obtains the information to be transmitted.

In a real application, this function should be replaced with the actual sensor, module, or application data source.

Encoding and transmitting data

```
message = json.dumps(data) + "\n"

encoded_message = message.encode("utf-8")

uart.write(encoded_message)
```

Converts the application data into JSON, encodes it as UTF-8 bytes, and sends it through UART.

Waiting for acknowledgement

```
response = uart.readline()
```

Reads a response until a newline character is received or the configured timeout expires.

If no response is received within the timeout, the application reports that no acknowledgement was received.

36.1 UART Receiver Application

The receiver application reads data transmitted by the external device.

It identifies complete messages using the newline delimiter, validates the JSON payload, and sends an acknowledgement.

37 Create *uart_receiver.py*

Create a Python file named *uart_receiver.py*.

The receiver performs the following operations:

1. Opens the configured serial interface.
2. Applies the UART communication settings.
3. Waits for incoming serial data.
4. Reads a complete message.

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

5. Decodes the received bytes.
6. Validates the JSON payload.
7. Displays the received information.
8. Sends an acknowledgement to the transmitter.

38 Complete Python Code

```
#!/usr/bin/env python3

"""
UART receiver application.

Receives JSON application data through a Linux
serial interface and sends an acknowledgement.
"""

import json
import serial

# UART configuration

SERIAL_PORT = "/dev/ttyS0"

BAUD_RATE = 115200

TIMEOUT = 2

def process_message(message):
    """Validate and process a received JSON message."""

    try:

        data = json.loads(message)

        if not isinstance(data, dict):

            raise ValueError(
                "Expected a JSON object"
            )

        print("\nReceived application data:")

        print(
            json.dumps(
                data,
                indent=4
            )
        )

        return {
            "status": "success",
            "message": "Data received successfully"
        }

    except (json.JSONDecodeError, ValueError) as error:
```

(continues on next page)

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

(continued from previous page)

```
        print("Invalid application data:", error)

        return {
            "status": "error",
            "message": "Invalid application data"
        }

def main():

    try:

        with serial.Serial(
            port=SERIAL_PORT,
            baudrate=BAUD_RATE,
            bytesize=serial.EIGHTBITS,
            parity=serial.PARITY_NONE,
            stopbits=serial.STOPBITS_ONE,
            timeout=TIMEOUT,
            xonxoff=False,
            rtscts=False,
            dsrdtr=False
        ) as uart:

            print("UART receiver started.")

            print("Serial port:", SERIAL_PORT)

            print("Baud rate:", BAUD_RATE)

            print("Waiting for incoming data...")

            while True:

                received_data = uart.readline()

                if not received_data:

                    continue

                try:

                    message = received_data.decode(
                        "utf-8"
                    ).strip()

                except UnicodeDecodeError:

                    print("Received invalid UTF-8 data.")

                    continue

                if not message:

                    continue
```

(continues on next page)

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

(continued from previous page)

```
                response = process_message(
                    message
                )

                response_data = (
                    json.dumps(response) + "\n"
                ).encode("utf-8")

                uart.write(response_data)

                uart.flush()

                print(
                    "Acknowledgement transmitted:",
                    response["status"]
                )

    except serial.SerialException as error:

        print("UART communication error:", error)

    except KeyboardInterrupt:

        print("\nUART receiver stopped.")

if __name__ == "__main__":

    main()
```

39 Explanation of Receiver Configuration

Opening the serial port

```
with serial.Serial(
    port=SERIAL_PORT,
    baudrate=BAUD_RATE,
    bytesize=serial.EIGHTBITS,
    parity=serial.PARITY_NONE,
    stopbits=serial.STOPBITS_ONE,
    timeout=TIMEOUT
) as uart:
```

Opens the selected serial device with the configured UART settings.

The transmitter and receiver must use compatible baud rate, data bits, parity, stop bits, and flow-control settings.

Reading incoming data

```
received_data = uart.readline()
```

Reads bytes until a newline character is received or the configured timeout expires.

The receiver waits for incoming data without continuously processing empty reads.

Decoding received data

```
message = received_data.decode("utf-8").strip()
```

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

Converts the received byte sequence into a UTF-8 string and removes surrounding whitespace.

Validating the message

```
data = json.loads(message)
```

Converts the received JSON string into a Python object.

The application also checks that the decoded value is a JSON object before processing it.

Sending acknowledgement

```
uart.write(response_data)

uart.flush()
```

Transmits a response through the UART interface.

The acknowledgement allows the transmitting application to determine whether the receiver processed the message successfully.

39.1 Application Deployment and Execution

The transmitter and receiver scripts must be available on their respective Linux devices.

Both devices must be connected through compatible UART interfaces.

40 Application Directory Structure

The application files can be organized as follows:

```
uart_application/
    |
    +-- uart_sender.py
    |
    +-- uart_receiver.py
```

The transmitter and receiver can be deployed on separate devices.

41 Verify the Serial Interface

On both devices, identify the available serial interface:

```
ls /dev/tty*
```

For example, if the selected interface is */dev/ttyS0*, verify it:

```
ls -l /dev/ttyS0
```

42 Verify UART Configuration

Execute:

```
stty -F /dev/ttyS0 -a
```

Confirm the required communication settings.

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

43 Configure the Hardware

Connect the transmitter and receiver:

Transmitter DeviceReceiver DeviceTXRXRXTXGNDGNDVerify the voltage levels and electrical interface before connecting the devices.

44 Start the Receiver

On the receiving Linux device, execute:

```
python3 uart_receiver.py
```

Expected output:

```
UART receiver started.
Serial port: /dev/ttyS0
Baud rate: 115200
Waiting for incoming data...
```

The receiver is now waiting for serial data.

45 Start the Transmitter

On the transmitting Linux device, execute:

```
python3 uart_sender.py
```

Expected output:

```
UART transmitter started.
Serial port: /dev/ttyS0
Baud rate: 115200
Waiting for communication...
```

The application begins sending JSON data every five seconds.

46 Stop the Applications

Press:

```
Ctrl + C
```

The application stops its communication loop and closes the serial interface.

46.1 Expected Output and Communication Validation

After both applications are running, the receiver should display the messages transmitted by the sender.

47 Transmitter Output

Example output:

```
UART transmitter started.
Serial port: /dev/ttyS0
Baud rate: 115200
```

(continues on next page)

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

(continued from previous page)

```
Waiting for communication...

Transmitted data:
{
    "device": "Linux_Device_01",
    "temperature": 28.5,
    "humidity": 62.0,
    "status": "normal"
}

Received acknowledgement:
{
    "status": "success",
    "message": "Data received successfully"
}
```

The acknowledgement confirms that the receiver returned a valid response.

48 Receiver Output

Example output:

```
UART receiver started.
Serial port: /dev/ttyS0
Baud rate: 115200
Waiting for incoming data...

Received application data:
{
    "device": "Linux_Device_01",
    "temperature": 28.5,
    "humidity": 62.0,
    "status": "normal"
}

Acknowledgement transmitted: success
```

The receiver displays the decoded application data and transmits the acknowledgement.

49 Communication Validation Checklist

Test case	Expected result
Serial interface identification	Correct serial device is identified.
Device permissions	Application has permission to access the serial port.
UART configuration	Both devices use compatible communication settings.
Hardware connection	TX, RX, and GND connections are correct.
Receiver initialization	Receiver opens the serial port successfully.
Transmitter initialization	Transmitter opens the serial port successfully.
Data transmission	Application data is transmitted through UART.
Data reception	Receiver obtains the complete message.
Message decoding	Received bytes are decoded correctly.
Data validation	Valid JSON data is processed successfully.
Acknowledgement	Receiver sends a response to the transmitter.
Application termination	Serial interfaces are closed correctly.

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

49.1 UART Loopback Testing

A loopback test is a simple method to verify whether a serial interface can transmit and receive data.

In a loopback test, the UART TX signal is connected to the same interface's RX signal.

Data transmitted through TX is routed back to RX.

50 Loopback Connection

For a compatible logic-level UART interface:

UART TX <-> UART RX

This test can be performed without connecting an external communication device.

Important: Perform a loopback test only when the selected interface is not connected to another active transmitter. Use the appropriate electrical connection for the interface being tested.

51 Loopback Test Using Python

Create a file named *uart_loopback.py*.

```
#!/usr/bin/env python3

"""
UART loopback test.

Connect the serial interface TX pin to its RX pin
using a suitable logic-level connection.
"""

import serial

SERIAL_PORT = "/dev/ttyS0"

BAUD_RATE = 115200

TEST_MESSAGE = b"UART_LOOPBACK_TEST\n"

def main():

    try:

        with serial.Serial(
            port=SERIAL_PORT,
            baudrate=BAUD_RATE,
            bytesize=serial.EIGHTBITS,
            parity=serial.PARITY_NONE,
            stopbits=serial.STOPBITS_ONE,
            timeout=2,
            xonxoff=False,
            rtscts=False,
            dsrdtr=False
        ) as uart:

            print("Starting UART loopback test.")

            uart.reset_input_buffer()
```

(continues on next page)

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

(continued from previous page)

```
            uart.write(TEST_MESSAGE)

            uart.flush()

            received_data = uart.readline()

            print(
                "Transmitted:",
                TEST_MESSAGE
            )

            print(
                "Received:",
                received_data
            )

            if received_data == TEST_MESSAGE:

                print("UART loopback test: PASS")

            else:

                print("UART loopback test: FAIL")

    except serial.SerialException as error:

        print("Serial port error:", error)

if __name__ == "__main__":

    main()
```

52 Execute the Loopback Test

Run:

```
python3 uart_loopback.py
```

Expected successful output:

```
Starting UART loopback test.
Transmitted: b'UART_LOOPBACK_TEST\n'
Received: b'UART_LOOPBACK_TEST\n'
UART loopback test: PASS
```

If the received data matches the transmitted data, the loopback test passes.

If the test fails, verify the TX/RX connection, serial-port permissions, port configuration, and whether another application is using the interface.

Note: A successful loopback test verifies basic local transmission and reception. It does not prove that the external device, cable, voltage levels, or complete communication path are working correctly.

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

52.1 UART Communication Monitoring

Linux provides utilities to inspect serial-port configuration and monitor communication.

53 Check Serial Configuration

Execute:

```
stty -F /dev/ttyS0 -a
```

This displays the current configuration of the selected serial interface.

54 Check Serial Device Ownership

Execute:

```
ls -l /dev/ttyS0
```

This displays the ownership and permissions of the serial device.

55 Identify Processes Using the Port

Execute:

```
sudo fuser -v /dev/ttyS0
```

This identifies processes that currently have the serial device open, if the *fuser* utility is installed.

56 Monitor Serial Data Using a Terminal Utility

If *minicom* is installed, it can be used to test serial communication manually.

Install it on Ubuntu or Debian-based systems:

```
sudo apt install minicom -y
```

Open the serial interface:

```
minicom -D /dev/ttyS0 -b 115200
```

The terminal utility opens the selected serial port using the specified baud rate.

Configure the data bits, parity, stop bits, and flow control to match the connected device.

To exit Minicom, use its exit key sequence, normally *Ctrl+A* followed by *X*.

Note: Do not run Minicom and the Python application on the same serial port simultaneously. Only one application should normally own the port at a time.

57 Inspect Kernel Messages

Execute:

```
dmesg | grep -i tty
```

This helps identify serial-device registration and related kernel messages.

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

57.1 Error Handling and Communication Reliability

UART applications must handle communication failures and unexpected data conditions.

Common problems include incorrect serial settings, disconnected hardware, incomplete messages, invalid data, and unavailable serial interfaces.

58 Serial Port Not Found

A serial-port error may occur when:

- The configured device file does not exist.
- The interface is not enabled or available.
- The expected USB-to-serial adapter is disconnected.
- The device has a different Linux device name.

Verify the available interfaces:

```
ls /dev/tty*
```

Update the application with the correct device path.

59 Permission Denied

The application may not have permission to access the serial device.

Check:

```
ls -l /dev/ttyS0
```

Verify that the application user belongs to the required group.

60 Incorrect Received Data

Unreadable or unexpected data may result from:

- Mismatched baud rates.
- Incorrect data bits.
- Incorrect parity.
- Incorrect stop bits.
- Electrical noise.
- Incorrect voltage levels.
- Wiring problems.

Verify the configuration on both devices and inspect the physical connections.

61 Communication Timeout

A timeout occurs when the application does not receive data within the configured time.

Possible causes include:

- The external device is not transmitting.
- TX and RX connections are incorrect.
- The receiver is using different UART settings.

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

- The external device requires a command before sending data.
- Hardware flow control is enabled incorrectly.

Check the hardware connection and communication requirements.

62 Invalid Message Format

Received data may be incomplete or may not match the expected application format.

The receiver should validate the message before processing it.

For applications requiring stronger reliability, consider using:

- Message length fields.
- Checksums or CRC.
- Sequence numbers.
- Acknowledgement and retry mechanisms.
- Explicit protocol version fields.

These mechanisms should be selected according to the external device's communication protocol.

63 Buffer and Data Loss Considerations

The receiving application must process data quickly enough to keep up with the incoming stream.

If data arrives faster than the application can process it, receive buffers may become full and data may be lost.

For continuous or high-speed data streams, applications should consider:

- Appropriate receive-buffer sizing.
- Efficient data parsing.
- Background reading or separate processing threads.
- Hardware flow control where supported.
- Application-level message framing.
- Monitoring communication errors.

The simple JSON example is intended for basic application messages, not unrestricted high-throughput serial streaming.

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

63.1 Troubleshooting

Issue	Possible cause	Recommended action
Serial port not found	Incorrect device path or unavailable interface	Check available interfaces using `ls /dev/tty*`.
Permission denied	Application user lacks access	Check device ownership and group membership.
No data received	Incorrect TX/RX wiring or external device not transmitting	Verify connections and the external device's operating status.
Garbled characters	Baud rate or framing mismatch	Verify baud rate, data bits, parity, and stop bits.
Communication timeout	No response from the external device	Check the external device, wiring, and response requirements.
Data transmitted but not received	TX/RX connections are incorrect	Connect TX to RX and RX to TX.
Loopback test failed	Incorrect loopback connection or port configuration	Verify the TX-to-RX connection and serial settings.
Port already in use	Another application or serial console owns the interface	Identify the process using `fuser`.
Intermittent communication	Electrical noise, loose wiring, or buffer overflow	Inspect wiring and review data rates and buffering.
Invalid JSON message	Incomplete or malformed application data	Validate the transmitted message and delimiter.
Hardware flow-control issue	RTS/CTS configuration does not match the wiring	Verify flow-control requirements and physical connections.
Device stops responding	External device reset, power issue, or protocol error	Check device power, status, and communication protocol.

64 Useful Diagnostic Commands

List serial interfaces:

```
ls /dev/tty*
```

Check serial-port settings:

```
stty -F /dev/ttyS0 -a
```

Inspect serial-device permissions:

```
ls -l /dev/ttyS0
```

Check kernel messages:

```
dmesg | grep -i tty
```

Identify processes using the port:

```
sudo fuser -v /dev/ttyS0
```

These commands help identify common serial-interface and application-level communication problems.

64.1 Security and Operational Considerations

UART interfaces may provide access to device consoles, diagnostic information, or application communication channels.

The following practices are recommended:

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization

- Restrict access to serial interfaces.
- Use appropriate Linux device permissions.
- Avoid exposing debug consoles in production environments unless required.
- Validate incoming data before processing it.
- Avoid transmitting passwords or other sensitive information through unprotected serial communication.
- Use suitable electrical protection and voltage-level conversion.
- Configure communication parameters according to the external device specifications.
- Avoid running multiple applications on the same serial port.
- Monitor communication failures and unexpected input.
- Implement appropriate timeout and recovery mechanisms.

UART itself does not provide encryption or authentication. If the communication carries sensitive information, the application must implement suitable security mechanisms or use a secure communication architecture.

64.2 Conclusion

UART communication provides a simple and widely supported method for exchanging serial data between Linux-based embedded systems and external devices.

By configuring the serial interface with compatible communication parameters, applications can transmit and receive data using the Linux serial device interface.

The Python implementation described in this document demonstrates how to open a serial port, configure UART settings, transmit structured application data, receive messages, validate data, and return acknowledgements.

The document also covers UART wiring, voltage compatibility, loopback testing, serial-port monitoring, error handling, and troubleshooting.

These procedures provide a practical foundation for integrating UART-based sensors, communication modules, controllers, and other external devices into Linux-based embedded applications.

64.3 References

- PySerial Documentation
- Python Serial Communication Library
- Linux TTY Documentation
- Linux Serial Programming Guide
- stty command manual
- fuser command manual

This document is property of PHYTEC. This can't be reproduced or communicated without written Authorization